Backend Services Documentation

The services layer contains all business logic. Each service is a standalone class or module imported by nodes, routes, or the enrichment worker.

PII Masking Service

backend/app/services/pii.py

Reversible PII masking using Microsoft Presidio NER + spaCy en_core_web_sm. Replaces detected entities with sequential tokens (, , ...) and stores a mapping for restoration after LLM calls. Masks subject and body only (not sender).

Method / Function	Parameters	Description
PIISanitizer.__init__	use_presidio: bool = True	Initialises Presidio AnalyzerEngine + spaCy. Falls back to regex-only mode if Presidio unavailable.
detect_pii	text: str → list[PIIEntity]	Returns list of detected PII spans (PERSON, EMAIL, PHONE, LOCATION, ORG, etc.) with start/end offsets.
mask_text	text: str → (masked: str, mapping: dict)	Replaces PII spans with tokens. Returns masked text + token-to-value mapping for later restoration.
restore_text	masked: str, mapping: dict → str	Substitutes all tokens back with original values.
strip_unresolved_tokens	text: str → str	Safety net: removes any tokens that have no mapping entry (LLM hallucinated token).
pii_sanitizer (module)	— (singleton)	Module-level singleton. Import pii_sanitizer and call mask_text() / restore_text() directly.

Tone DNA Service

backend/app/services/tone_dna.py

Builds a 8-feature stylometric profile from a user's sent email history. Profile is stored in tone_profile table and injected as a system prefix in every draft.

Method / Function	Parameters	Description
_avg_sentence_length	text: str → float	Average word count per sentence across the email body.
_formality_score	text: str → float	Ratio of formal markers (therefore, regarding, sincerely) vs informal (hey, wanna, gonna).
_greeting_patterns	texts: list[str] → list[str]	Extracts most common greeting phrases (Hi X, Dear X, Hello X).
_signoff_patterns	texts: list[str] → list[str]	Extracts most common sign-off phrases (Thanks, Best, Regards).
_contraction_rate	text: str → float	Fraction of words that are contractions (it's, don't, we're).
_bullet_preference	text: str → float	Fraction of emails using bullet lists or numbered lists.
_emoji_rate	text: str → float	Average emoji count per email.
_top_vocabulary	texts: list[str], n: int → list[str]	Most frequently used non-stopword tokens.
build_profile	emails: list[str] → dict	Orchestrates all 8 feature extractors. Returns profile dict with sample_size.
ToneDNAService.ingest_and_build	account_id: str, graph_client → dict	Fetches sent emails via provider, calls build_profile(), saves to DB.

ToneDNAService.load_profile	`account_id: str → dict|None`	Loads stored profile from tone_profile table.
ToneDNAService.save_profile	`account_id: str, profile: dict → None`	Upserts profile to DB.

Draft Service

`backend/app/services/draft_service.py`

Generates AI draft replies using GPT-4o, injecting Tone DNA style prefix and RAG precedents.

Method / Function	Parameters	Description
`_get_llm_client`	→ AzureChatOpenAI|ChatGroq	Returns cached LLM instance (Azure first, Groq fallback).
`_get_clean_name`	sender: str → str	Extracts first name from "John Smith " format.
`_get_user_display_name`	email: str → str	Derives first name from email prefix.
`DraftService.generate_draft`	email_text, style, sender, subject, current_user_email, account_id → (draft, citations)	Main method. Loads Tone DNA profile for account, fetches RAG precedents, builds system prompt with style prefix, calls GPT-4o. Styles: standard | formal | indepth.
`DraftService.generate_compose`	recipient, subject, context, account_id → str	Generates a new outbound email (not a reply). Still uses Tone DNA prefix for style consistency.

Commitment Service

`backend/app/services/commitments.py`

Extracts action items and deadlines from email text using GPT-4o structured output. Caches results by email_id. Falls back to regex when LLM is unavailable.

Method / Function	Parameters	Description
`CommitmentService._get_llm_client`	→ (client, model_name)	Azure → Groq → None fallback chain.
`CommitmentService._fallback_extract`	masked_email_text: str → list	Regex-based extraction: looks for patterns like "by [date]", "please [verb]", "deadline:".
`CommitmentService.extract`	masked_email_text, thread_summary, email_id → list[CommitmentItem]	Main method. Checks cache first (by email_id or text hash). Calls GPT-4o with few-shot examples for structured CommitmentItem output. Each item: id, commitment, deadline, confidence, approved, conflict_badge.
`CommitmentService.confirm_commitment`	commitment_id, email_id, account_id → bool	Creates calendar event for a confirmed commitment via provider API.

RAG / Vector Retrieval Service

`backend/app/services/rag.py`

Local vector index using ChromaDB (file-based JSON). Embeds with text-embedding-ada-002. Cosine similarity threshold 0.78 for retrieval.

Method / Function	Parameters	Description
`mask_pii`	text: str → str	Lightweight regex PII redaction before indexing sent emails.
`cosine_similarity`	a: list, b: list → float	Pure-Python dot product / magnitude similarity.
`EmbeddingProvider.embed`	text: str → list[float]	Calls Azure text-embedding-ada-002. Falls back to _deterministic() if unavailable.

EmbeddingProvider._deterministic	`text: str → list[float]`	Hash-based pseudo-embedding for offline/test mode.
ChromaDBIndex.index	`documents: list[dict] → None`	Batch indexes documents to JSON store.
ChromaDBIndex.search	`vector: list, top_k: int, threshold: float → list[PrecedentItem]`	Cosine similarity search. Returns items above threshold, ranked by score.
ChromaDBIndex.upsert	`document: dict → None`	Add or update single document.
ChromaDBIndex.trim	`max_size: int → None`	Removes oldest documents when index exceeds max_size.
RetrievalService.retrieve	`query: str, account_id: str, top_k: int → list[PrecedentItem]`	Embeds query, searches account-scoped index, returns precedents.

Calendar Service

`backend/app/services/calendar_service.py`

Deterministic (no LLM) calendar conflict detection. Compares commitment deadlines against fetched calendar events.

Method / Function	Parameters	Description
check_calendar_conflict	commitment_deadline: datetime, events: list[CalendarEvent] → (bool, str)	Returns (has_conflict, conflict_detail). A conflict exists when commitment_deadline falls within any event time window ± 30-minute buffer.

Sync Service

`backend/app/services/sync_service.py`

Manages the server-side mailbox mirror. Supports full backfill (first sync) and cursor-based delta sync for incremental updates.

Method / Function	Parameters	Description
SyncService._apply_changes	account_id, folder, result → (new_count, updated, removed)	Applies upserts and tombstones from a provider response to the DB.
SyncService.backfill	account: OAuthAccount, folder: str → dict	Full enumeration on first sync. Fetches all emails, stores envelopes, saves initial delta cursor. Ensures webhook subscription.
SyncService.delta_sync	account: OAuthAccount, folder: str → dict	Incremental sync using stored delta cursor. Falls back to backfill if cursor missing or expired.

Microsoft Graph Client

`backend/app/services/graph.py`

Microsoft Graph API client (~90KB). Handles Outlook mail, calendar, and delta sync. USE_MOCK_GRAPH=true returns rich mock data for development.

Method / Function	Parameters	Description
GraphClient.__init__	access_token, refresh_token, use_mock: bool	Initialises with token credentials and mock flag.
GraphClient.get_inbox_emails	folder, limit → list[EmailItem]	MOCK: returns 10 emails (CRITICAL×2, HIGH×3, MEDIUM×3, LOW×2). REAL: calls /me/mailFolders/{folder}/messages.
GraphClient.get_calendar_events	days_ahead: int → list[CalendarEvent]	MOCK: returns 5 events overlapping email deadlines. REAL: calls /me/calendar/calendarView.
GraphClient.fetch_sent_emails	limit: int → list[str]	MOCK: returns 55 emails (20 rich VP/Director persona + 35 fillers). REAL: calls /me/mailFolders/SentItems/messages.
GraphClient.get_email_body	email_id: str → str	Fetches full HTML/text body for an email.
GraphClient.send_reply	email_id, body → None	Sends reply via Graph /me/messages/{id}/reply.
GraphClient.get_delta	folder, delta_link → dict	Fetches delta changes using provided deltaLink cursor.
GraphClient._days_ago	n: int → str	Helper: ISO date string N days in the past.

GraphClient.ensure_subscription	`webhook_url, account_id → str`	Creates or renews Graph change notification subscription.

Gmail Client

`backend/app/services/gmail.py`

Gmail API client (~61KB). Handles inbox, sent mail, and Pub/Sub watch. Mock mode returns matching emails for dev.

Method / Function	Parameters	Description
GmailClient.__init__	credentials, use_mock: bool	Initialises Google OAuth2 credentials.
GmailClient.get_inbox_emails	limit: int → list[EmailItem]	MOCK: 10 emails (gmail-1 to gmail-10) with matching triage profiles. REAL: Gmail messages.list + get per message.
GmailClient.fetch_sent_emails	limit: int → list[str]	MOCK: 50 emails (15 rich + 35 fillers). REAL: Sent label messages.
GmailClient.get_email_body	message_id: str → str	Decodes base64url email body from Gmail API.
GmailClient.send_reply	thread_id, body, to → None	Sends reply via Gmail messages.send.
GmailClient.watch	topic_name: str → dict	Starts Gmail Push notifications via Cloud Pub/Sub watch.
GmailClient.get_history	history_id: str → dict	Fetches history.list for delta sync since historyId cursor.
GmailClient._mock_inbox	→ list	Returns 10 hardcoded realistic inbox emails with bodies, senders, and deadlines.

Session Service

`backend/app/services/session_service.py`

Manages user sessions and Quick Login tokens. Session tokens are SHA-256 hashed before storage.

Method / Function	Parameters	Description
SessionService.create_session	user_id: UUID → str	Generates secure random token, SHA-256 hashes it, stores UserSession row with 24h TTL. Returns raw token.
SessionService.validate_session	token: str → UserSession|None	Hashes token, looks up UserSession, checks expiry.
SessionService.extend_session	token: str → None	Updates last_seen_at and optionally extends expires_at.
SessionService.revoke_session	token: str → None	Deletes UserSession row (logout).
SessionService.create_quick_login	user_id, device_id → str	Creates 7-day QuickLoginToken for auto-resume.
SessionService.validate_quick_login	token: str → QuickLoginToken|None	Validates long-lived token, checks status and expiry.

API Routes Documentation

Main Routes (backend/app/api/routes.py)

`backend/app/api/routes.py`

OAuth callbacks, inbox listing, email operations. The _finish_oauth_connect() function is the core post-OAuth flow (upsert user, create session, set cookie).

Method	Path	Auth	Description
GET	`/api/auth/status`	Session	Returns authenticated user info, default account, and list of connected accounts.
GET	`/api/auth/microsoft`	None	Initiates Microsoft OAuth — redirects to Azure Entra ID consent screen.
GET	`/api/auth/microsoft/callback`	None	Exchanges auth code, upserts user/account, sets mm_session cookie.
GET	`/api/auth/google`	None	Initiates Google OAuth — redirects to Google consent screen.
GET	`/api/auth/google/callback`	None	Exchanges code, upserts user/account, sets mm_session cookie.
POST	`/api/auth/logout`	Session	Revokes session, clears cookie.
POST	`/api/inbox`	Session	Lists inbox emails. Joins mailbox_message + enrichment. Supports folder, sort, filter, pagination.
GET	`/api/inbox/{email_id}`	Session	Fetches full email body + metadata from provider.
POST	`/api/draft`	Session	Calls DraftService.generate_draft() with specified style.
POST	`/api/reply`	Session	Sends reply via provider API (Graph.send_reply or Gmail.send_reply).
POST	`/api/compose`	Session	Generates and optionally sends a new outbound email.
POST	`/api/commitments/extract`	Session	Calls CommitmentService.extract() on provided email text.
POST	`/api/commitments/confirm`	Session	Creates calendar event for confirmed commitment.
PUT	`/api/priority/{email_id}`	Session	Records priority override — sets enrichment + mailbox_message to done state.
PUT	`/api/tone-dna/build`	Session	Triggers ToneDNAService.ingest_and_build() for the default account.
GET	`/api/tone-dna/preview`	Session	Returns current Tone DNA profile for display.
GET	`/api/user/accounts`	Session	Lists all connected OAuthAccounts for the user.

Agent Routes (backend/app/api/agent_routes.py)

`backend/app/api/agent_routes.py`

LangGraph pipeline API endpoints. Supports full pipeline, streaming (SSE), triage-only, enrichment-only, and human-in-the-loop approval.

Method	Path	Auth	Description
GET	`/api/agent/health`	None	Returns LLM provider status and pipeline readiness.
POST	`/api/agent/process`	Session	Runs full 6-node pipeline synchronously. Returns complete enriched state.
POST	`/api/agent/stream`	Session	Streaming pipeline via SSE. Yields JSON events per node completion.
POST	`/api/agent/triage`	Session	Runs ingest + triage nodes only (Phase 1). Returns axes, priority, score.
POST	`/api/agent/enrich`	Session	Runs commitment + calendar + RAG nodes (Phase 2). Returns commitments, draft, precedents.
POST	`/api/agent/batch`	Session	Triage multiple emails in parallel (uses asyncio.gather, max TRIAGE_MAX_WORKERS concurrent).
GET	`/api/agent/approve/{email_id}`	Session	Checks approval status for gated email.
POST	`/api/agent/approve/{email_id}`	Session+Token	Approves gated email. Requires X-Approval-Token header.

Sync Routes (backend/app/api/sync_routes.py)

Method	Path	Auth	Description
POST	`/webhooks/graph`	client_state	Receives Microsoft Graph change notifications. Verifies client_state, triggers delta_sync.
POST	`/webhooks/gmail`	token param	Receives Gmail Pub/Sub push. Verifies ?token=, decodes historyId, triggers delta_sync.
POST	`/api/subscriptions/ensure`	Session	Creates Graph webhook subscription for authenticated account.
POST	`/api/subscriptions/renew`	Admin	Renews expiring Graph subscriptions (cron job target).

Waitlist Routes (backend/app/api/waitlist_routes.py)

Method	Path	Auth	Description
POST	`/api/waitlist/join`	None	Adds email to waitlist (status=pending). Returns position.
GET	`/api/waitlist/status`	None	Check if email is approved (for login gate).
POST	`/api/admin/waitlist/approve/{email}`	X-Admin-Token	Approves waitlisted email — allows sign in.
POST	`/api/admin/waitlist/reject/{email}`	X-Admin-Token	Rejects waitlisted email.
GET	`/api/admin/feedback`	X-Admin-Token	Lists all product feedback submissions.

Other Routes

Method	Path	Auth	Description
GET	`/api/demo/login`	None	Returns HTML one-click demo login page.
POST	`/api/demo/login`	None	Creates session for demo@mailmind.app. No waitlist gate.
POST	`/api/pii/preview`	Session	Runs PII detection on provided text and returns masked output + entity list.
POST	`/api/feedback`	Session	Stores product feedback (rating, category, message).
GET	`/api/health`	None	Liveness probe (always 200).
GET	`/api/ready`	None	Readiness probe (checks DB connection).
GET	`/api/metrics`	None	Prometheus-format metrics (requests, latency, SLA).
GET	`/api/compliance/audit/{email_id}`	Session	Returns audit trail for an email.
DELETE	`/api/compliance/data/{user_id}`	Session	GDPR right-to-erasure — deletes all user data.

Frontend Documentation

Next.js 15 App Router frontend with TypeScript, Tailwind CSS, GSAP animations, and SSE streaming.

App Pages (frontend/app/)

Page File	Route	Purpose
app/page.tsx	/	Landing page with WebGL hero, GSAP animations, 6 feature cards, pipeline visualizati
app/layout.tsx	/	Root layout: Geist fonts, global CSS, SEO metadata, error boundary, Vercel Analytics
app/dashboard/page.tsx	/dashboard	Main inbox dashboard: email list + detail panel side-by-side. Auth guard.
app/login/page.tsx	/login	OAuth login page with Google and Microsoft provider buttons
app/admin/page.tsx	/admin	Admin panel: waitlist approval, feedback list. Requires admin token.
app/privacy/page.tsx	/privacy	PII masking demo and privacy information page
app/terms/page.tsx	/terms	Terms of service legal document

Key Components

Component	Location	Purpose
EmailList	components/inbox/	Main inbox list with pagination, sort, filter chips, priority badges, SSE triage stream
EmailListItem	components/inbox/	Single email row: sender, subject, snippet, priority badge, score bar, action buttons
EmailDetail	components/detail/	Full email view: HTML body (sanitized), triage explainer, commitments gate, draft
DraftPanel	components/detail/	Draft generation UI: style selector (standard/formal/indepth), edit area, send buttor
CommitmentGate	components/commitments/	Modal for approving commitments: deadline, confidence, conflict badge, approve/c
PrecedentList	components/detail/	Shows top 3 RAG-retrieved similar past emails with similarity scores
PipelineVisualization	components/pipeline/	Interactive 6-node pipeline diagram for landing page presentation mode
HeroCanvas	components/landing/	Three.js WebGL animated hero — particle pipeline visualization
WaitlistForm	components/landing/	Email waitlist signup with success/error states
Header	components/layout/	Top navigation: logo, account switcher, logout, settings
Sidebar	components/layout/	Folder navigation: INBOX, SENT, DRAFTS, priority filters
PriorityBadge	components/inbox/	Color-coded CRITICAL/HIGH/MEDIUM/LOW badge chip
TriageScoreBar	components/inbox/	Visual bar showing composite score 0-100
PriorityOverrideMenu	components/inbox/	Dropdown to manually change email priority or mark done
FilterMenu	components/inbox/	Priority filter (ALL/CRITICAL/HIGH/MEDIUM/LOW) with counts
ComposeWindow	components/inbox/	Full compose window for new email drafts
ErrorBoundary	components/shared/	React error boundary with fallback UI and error reporting
FeedbackModal	components/shared/	Product feedback form: rating, category, message

Custom React Hooks

Method / Function	Parameters	Description
useEmails	`accountId, defaultFolder, pageSize`	Manages entire inbox state: emails list, total, pagination (page index, next/prev), sort key, filters (priority, folder), selected email. Calls /api/inbox with params.
useEmailDetail	`email, enabled, currentUserEmail, onTriageEnriched`	Two-phase loading hook. Phase 1: reads triage from email object instantly. Phase 2: calls /api/agent/enrich to load commitments + draft + precedents. Caches in localStorage.
useAuthFlow	–	Manages OAuth flow state: loading, error, provider. Calls /api/auth/status on mount.
useInboxCache	`accountId`	localStorage-based inbox cache: stores last 100 email envelopes with TTL.

API Client (frontend/lib/api.ts)

`frontend/lib/api.ts`

Fetch wrapper for all backend calls. Automatically includes credentials (cookies). Base URL from NEXT_PUBLIC_API_URL env var or relative (/api/...).

Method / Function	Parameters	Description
apiFetch	`input: RequestInfo, init?: RequestInit → Response`	Wraps native fetch with credentials:"include" and base URL resolution.
enrichEmail	`payload: EmailPayload → Enrichment`	POST /api/agent/enrich — Phase 2 enrichment.
triageEmail	`payload → TriageResult`	POST /api/agent/triage — Phase 1 triage.
triageWithRetry	`payload, maxRetries: number → TriageResult`	Triage with exponential backoff (3 retries, 1s/2s/4s).
generateEmailDraft	`payload: DraftRequest → DraftResponse`	POST /api/draft — generate AI draft reply.
sendEmailReply	`payload: ReplyRequest → void`	POST /api/reply — send reply via provider.
fetchMailboxMessage	`emailId: string → MailboxMessage`	GET /api/inbox/{emailId} — fetch full email body.
fetchAttachments	`emailId: string → Attachment[]`	GET /api/inbox/{emailId}/attachments.